

Lab 1 : DevSecOps Fundamentals & SDLC Security Integration

Zawad Hossain

Cybersecurity Architecture, Design and SSD - MCSC-6002-RNA

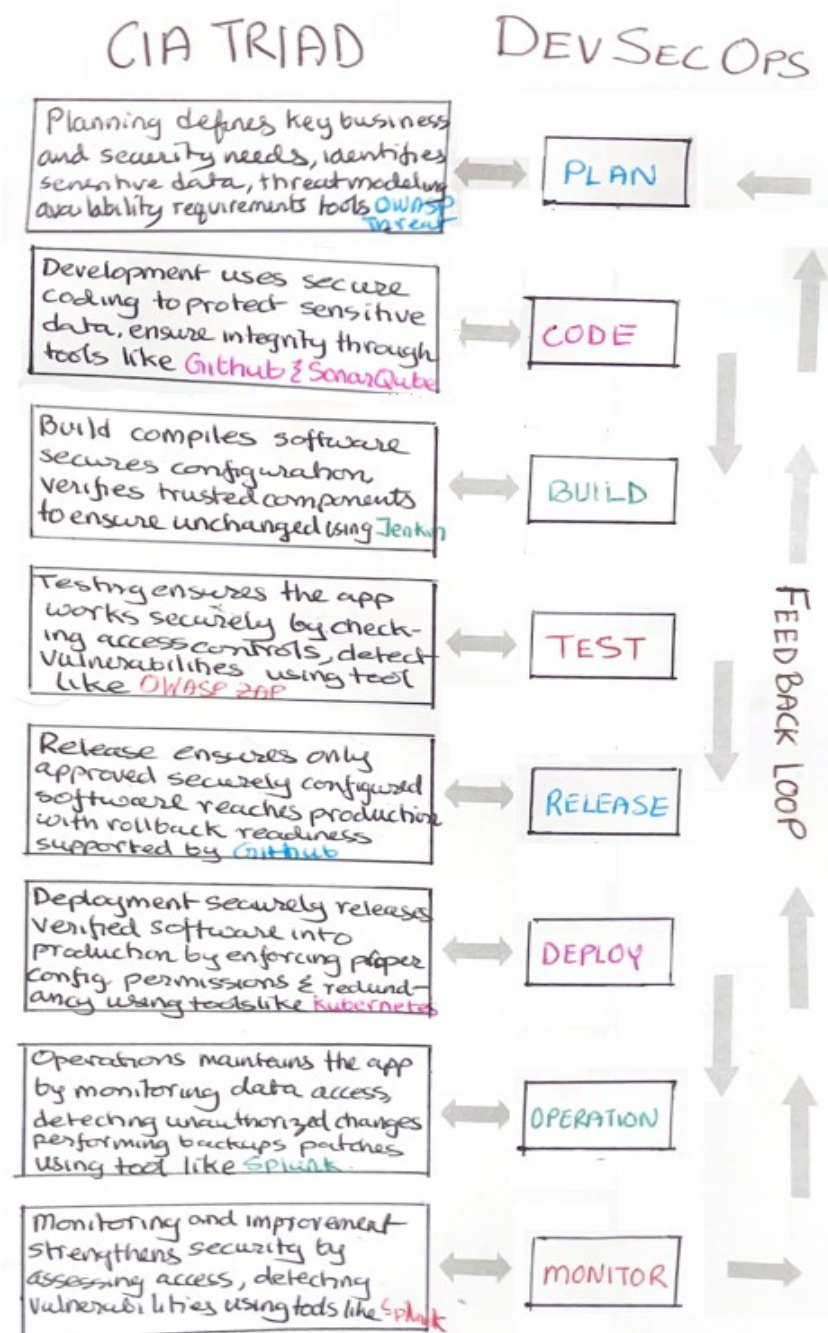
Professor Timothy Wong

24th July 2026

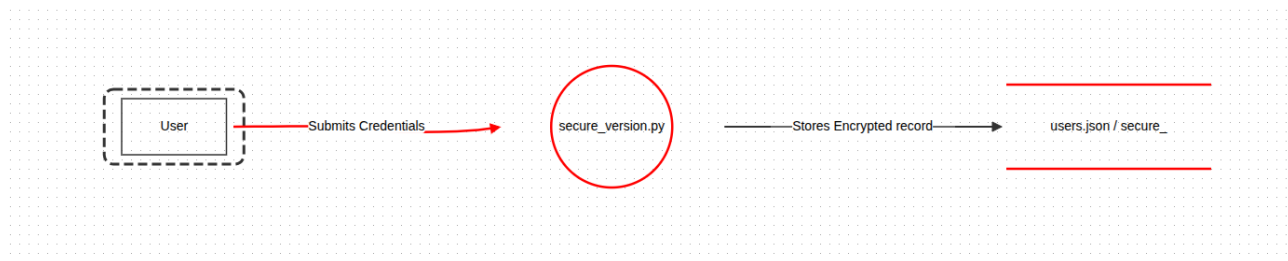
Background

This lab introduces the fundamentals of DevSecOps by examining the Software Development Life Cycle (SDLC) through a security-focused lens and applying the Confidentiality, Integrity, and Availability (CIA) triad in practice. We will build a two-phase Python application—first intentionally insecure, then redesigned using secure coding principles—to explore how hashing, data-integrity verification, logging, and tampering detection strengthen software security. Through this hands-on exercise, we will learn how security is integrated into development workflows and how DevSecOps transforms a vulnerable application into a resilient one. Task 1 – CIA Triad mapping to DevSecOps phases.

Task 1 – DevSecOps phases mapping to CIA Triad



Task 2 – Insecure Coding



```
import json

#Hardcoded secret key sitting directly in the source code
SECRET_KEY = "mysecretkey123"

#Simple encryption easily reversible with a fixed key
def fake_encrypt(password):
    return "".join(chr(ord(c) ^ len(SECRET_KEY)) for c in password)

#No input validation
def add_user_insecure():
    username = input("Username: ")
    password = input("Password: ")
    email = input("Email: ")

    encrypted_password = fake_encrypt(password)

#File handling exception missing
    with open("users.json", "r") as f:
        users = json.load(f)

    users.append({"username": username, "password": encrypted_password, "email":
email})

#No integrity value (hash) is computed or stored anywhere.
# If someone edits this file later there is no way for the program to detect
that it happened
    with open("users.json", "w") as f:
        json.dump(users, f, indent=4)

    print("User added (insecurely).")
```

Simple encryption, no input validation, handling exception

```

#(violates DRY - Don't Repeat Yourself):instead of reusing one shared function.
def print_all_users_insecure():
    with open("users.json", "r") as f:
        users = json.load(f)

    for u in users:
        #Sensitive data (even the "encrypted" password) is printed straight to the
        #console.
        #In real deployments this ends up in terminal history, CI logs, or crash reports
        print(u["username"], u["password"], u["email"])

if __name__ == "__main__":
    add_user_insecure()
    print_all_users_insecure()

```

Printing in the consoles

```

zh26@ZeeDell:~/Documents/Humber MSc/6002/assignments/Lab/1/Submission$ python3 insecure_version.py
Username: frank
Password: FrankPass2024!
Email: frank@example.com

```

```

frank H|o`e^o}}<><:/ frank@example.com

```

Secure Version

```
GNU nano 7.2 secure_version.py
0 |
1 |
2 | def secure_store(input_file="users.json", output_file="secure_users.json"):
3 |     users = load_users(input_file)
4 |     secure_users = []
5 |     reference_hashes = {}
6 |
7 |     for user in users:
8 |         username = user["username"]
9 |         password = user["password"]
0 |         email = user["email"]
1 |
2 |     #Integrity hash computed and stored per record,
3 |     #any later change to the data can be detected.
4 |         record_hash = compute_hash(username, password, email)
5 |         reference_hashes[username] = record_hash
6 |
7 |         secure_users.append({
8 |             "username": username,
9 |             "password": encrypt_password(password),
0 |             "email": email,
1 |             "integrity_hash": record_hash
2 |         })
3 |
4 |     save_users(secure_users, output_file)
5 |     with open("reference_hashes.json", "w") as f:
6 |         json.dump(reference_hashes, f, indent=4)
7 |     logging.info(f"secure_store: processed {len(secure_users)} records into {output_file}")
8 |
9 |
0 | #Never prints raw or encrypted passwords
1 | def print_all_users_secure(secure_file="secure_users.json"):
2 |     users = load_users(secure_file)
3 |     for u in users:
4 |         print(u["username"], "*****", u["email"])
5 |
6 |
7 |
8 |
9 |
10 |
11 |
12 |
13 |
14 |
15 |
16 |
17 |
18 |
19 |
20 |
21 |
22 |
23 |
24 |
25 |
26 |
27 | def verify_integrity(secure_file="secure_users.json"):
28 |     """Checks secure_users.json against its own embedded hashes."""
29 |     users = load_users(secure_file)
30 |     print(f"\n--- Verifying integrity of '{secure_file}' ---")
31 |     for user in users:
32 |         decrypted = decrypt_password(user["password"])
33 |         recomputed = compute_hash(user["username"], decrypted, user["email"])
34 |         if recomputed == user["integrity_hash"]:
35 |             print(f" [PASS] {user['username']}")
36 |             logging.info(f"Integrity check PASSED for user '{user['username']}'")
37 |         else:
38 |             print(f" [FAIL] {user['username']} (tampering detected)")
39 |             logging.warning(f"INTEGRITY VIOLATION detected for user '{user['username']}'")
40 |
```

```

zh26@ZeeDell:~/Documents/Humber MSc/6002/assignments/Lab/1/Submission$ python3 secure_version.py
=====
PHASE 2 - TASK 2: Secure Data Storage
=====
alice ***** alice@example.com
bob ***** bob@example.com
carol ***** carol@example.com
dave ***** dave@example.com
eve ***** eve@example.com

--- Verifying integrity of 'secure_users.json' ---
[PASS] alice
[PASS] bob
[PASS] carol
[PASS] dave
[PASS] eve

```

Improved version, hiding the password with encryption

Task 3 – Threat Analysis

Submitted as a separate document

Task 4 – Secured Coding

```

GNU nano 7.2                                secure_version.py
1 import json
2 import hashlib
3 import logging
4 from cryptography.fernet import Fernet
5
6 KEY_FILE = "secret.key"
7 LOG_FILE = "security_audit.log"
8
9 #Instead of printing confidential data on the console
10 #All integrity checks are now written to a persistent log file
11 # (with a timestamp) In a real system this log file is what a SIEM tool like Splunk
12 # would monitor to alert a security team the moment tampering is detected.
13 logging.basicConfig(
14     filename=LOG_FILE,
15     level=logging.INFO,
16     format="%(asctime)s [%(levelname)s] %(message)s"
17 )
18
19 #Key is no longer hardcoded in the source code. its stored in own file,
20 # which in a real deployment would be restricted with
21 # file permissions or moved to a secrets manager.
22 def load_or_create_key():
23     try:
24         with open(KEY_FILE, "rb") as f:
25             return f.read()
26     except FileNotFoundError:
27         key = Fernet.generate_key()
28         with open(KEY_FILE, "wb") as f:
29             f.write(key)
30     return key
31
32

```



```
3 #Real encryption (AES-based, via Fernet).
4 KEY = load_or_create_key()
5 fernet = Fernet(KEY)
6 def encrypt_password(password):
7     return fernet.encrypt(password.encode()).decode()
8
9 def decrypt_password(encrypted_password):
0     return fernet.decrypt(encrypted_password.encode()).decode()
1
2
3 def compute_hash(username, password, email):
4     record_string = username + password + email
5     return hashlib.sha256(record_string.encode()).hexdigest()
6
7 #Basic input validation - rejects empty fields, short
8 # passwords, and clearly malformed emails.
9 def validate_input(username, password, email):
0     if not username or not password or not email:
1         return False, "Username, password, and email cannot be empty."
2     if len(password) < 8:
3         return False, "Password must be at least 8 characters long."
4     if "@" not in email or "." not in email:
5         return False, "Email address does not look valid."
6     return True, ""
7
8 # Implementing DRY principle. Handles a missing or corrupt file gracefully instead of crashing.
9 def load_users(filename):
0     try:
1         with open(filename, "r") as f:
2             return json.load(f)
3     except FileNotFoundError:
4         logging.warning(f"{filename} not found - starting with empty list.")
5         return []
6     except json.JSONDecodeError:
7         logging.error(f"{filename} is not valid JSON.")
8         return []
9
```

```

41
42 def check_for_tampering(reference_hash_file="reference_hashes.json",
43                        file_to_check="users_tampered.json"):
44     """
45     Task 4: compares a possibly-tampered file (users_tampered.json)
46     against the ORIGINAL reference hashes saved by secure_store().
47     Every result - pass or fail - is written to security_audit.log,
48     simulating what a log-monitoring tool (e.g. Splunk) would record.
49     """
50     with open(reference_hash_file, "r") as f:
51         reference_hashes = json.load(f)
52
53     users = load_users(file_to_check)
54
55     print(f"\n--- Checking '{file_to_check}' against original hashes ---")
56     logging.info(f"Starting tamper check on '{file_to_check}'")
57     tampered_found = False
58
59     for user in users:
60         username = user["username"]
61         password = user["password"]
62         email = user["email"]
63
64         current_hash = compute_hash(username, password, email)
65         original_hash = reference_hashes.get(username)
66
67         if current_hash == original_hash:
68             print(f"    [PASS] {username}: no tampering detected")
69             logging.info(f"Tamper check PASSED for user '{username}'")
70         else:
71             print(f"    [FAIL] {username}: TAMPERING DETECTED")
72             logging.warning(f"TAMPERING DETECTED in '{file_to_check}' for user '{username}' "
73                            f"- expected hash {original_hash}, got {current_hash}")
74             tampered_found = True
75

```

```

    if tampered_found:
        print(f"\nResult: >>> Tampered record(s) found in {file_to_check} <<<")
        logging.warning(f"Tamper check complete: tampering found in {file_to_check}")
    else:
        print(f"\nResult: No tampering found.")
        logging.info(f"Tamper check complete: no tampering found in {file_to_check}")

    return tampered_found

```



```
=====
PHASE 2 - TASK 4: Data Integrity Verification (Tamper Detection)
=====

--- Checking 'users_tampered.json' against original hashes ---
[PASS] alice: no tampering detected
[FAIL] bob: TAMPERING DETECTED
[PASS] carol: no tampering detected
[PASS] dave: no tampering detected
[PASS] eve: no tampering detected

Result: >>> Tampered record(s) found in users_tampered.json <<<

[i] Full audit trail written to security_audit.log - open it to see every
    PASS/FAIL event with a timestamp, as a log-monitoring tool would.
Username: cr7
Password: abc123
Email: cr7@uefa.com
[REJECTED] Password must be at least 8 characters long.
Username: cr7
Password: Balandor!2026
Email: cr7@uefa.com
[OK] User 'cr7' added securely.
zh26@ZeeDell:~/Documents/Humber MSc/6002/assignments/Lab/1/Submission$
```

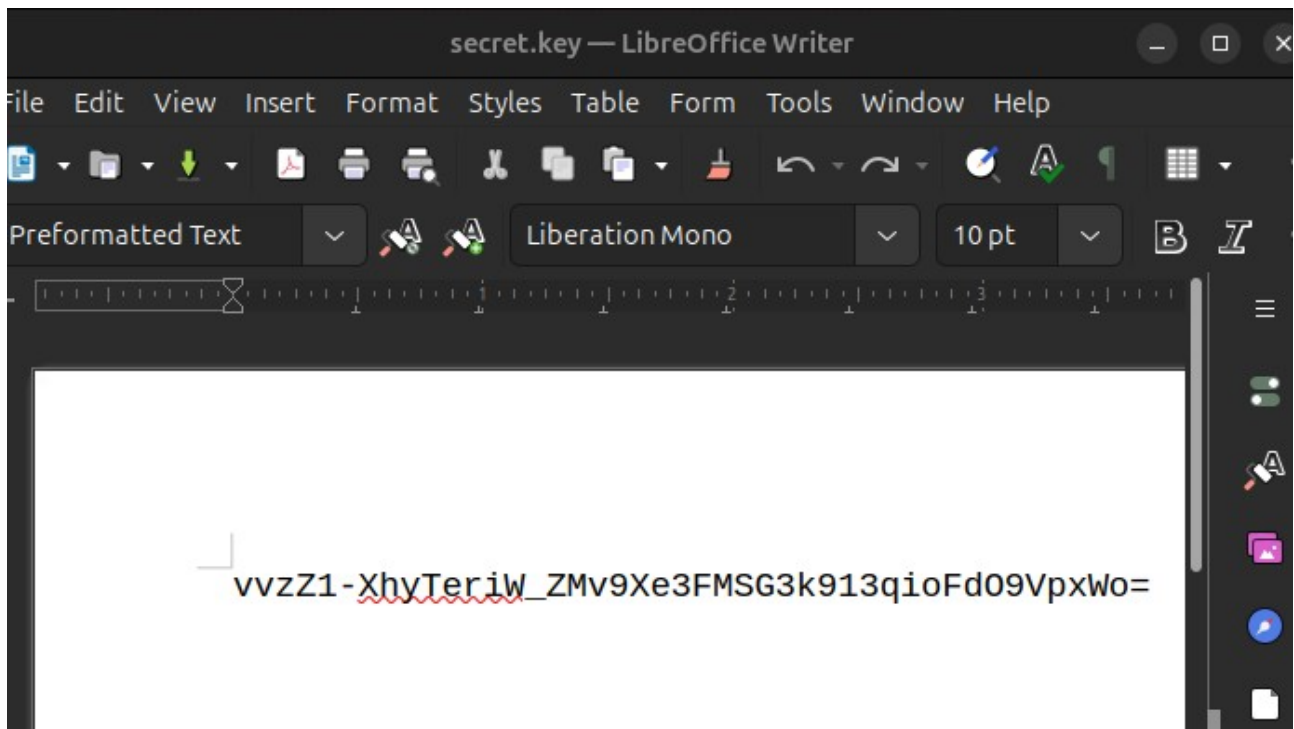
Tele

Secure coding principles and proper validation integrity checks

```
Open  ~  secure_users.json  ~/Documents/Humber MSc/6002/assignments/Lab/1/Submission  [Icons]
Empty Document  users.json  secure_users x  insecure_version  security_audit.log  reference_hash

[
  {
    "username": "alice",
    "password": "qAAAAABqYD5p0dKRLPk2QSNpIbpLiq1XCB10j2ZuEzaFJ_gpvsozrQgExZixW4qLt7bI5qa3IcjsxpTyDhdAd5RVdvsr1SkbFA==",
    "email": "alice@example.com",
    "integrity_hash": "7307e492e03853027c9257c3c51588ca2973c359a56a4f8f96388bd787afdf9"
  },
  {
    "username": "bob",
    "password": "qAAAAABqYD5p0C273lALSA2u0qt3cBHTCt1b4yBCrU_h1MSyHqTHYoe7_Zuc1nRGbGBkBrI44fu2fWmBrIl9N7IMlswfR-6TCA==",
    "email": "bob@example.com",
    "integrity_hash": "c19d5fe25c6b0ea48bad48e5d31de16cefc341486dce3f0b6560d88f5ceeeed"
  },
  {
    "username": "carol",
    "password": "qAAAAABqYD5psh03p5NSmX_GriIG0TfN083Kl0as3s5jX3wUJKPx0AsnFbo0bSqJs7m2jbaf3R0L4CfAK8FT0dB-r6nPta8qUw==",
    "email": "carol@example.com",
    "integrity_hash": "cab2ccacdfc996b4f06f1e285e9b8384cec50e8fee79be655e4c99eb0b192f19"
  },
  {
    "username": "dave",
    "password": "qAAAAABqYD5p8LVwhuWuaJR9z4tg8k5c_XxXXs7hJ4v4u7ktrB47K9wTmkzFKrk9nXkb8fj0Sj0kGff00VUH2qcaeA00-ylvfA==",
    "email": "dave@example.com",
    "integrity_hash": "eb3c2d89ec746c24a737dbe909e76d5c8c6efa8b5132ef68f0a90e5280279563"
  },
  {
    "username": "eve",
    "password": "qAAAAABqYD5p5bzfp2qztfVixEGhT9ExnnnU-fblqWAbg2t82FzJ0JJMV0fZ6o_Dm2h15cuG803YXKqJe009vp83P0qXs5513w==",
    "email": "eve@example.com",
    "integrity_hash": "b5d8a7a54e65a81c9352826f62e940ecc80ab1911aa3a1c18f3c1ef8ab717003"
  },
  {
    "username": "cr7",
    "password": "qAAAAABqYD5prlWtl0WMLD1qBPZyWwekXmyTKshFAfhmpIHVut01m_3WhhAviuP1r40yQ8JwquA019afq6_jTvox6JvoSrL2Zw==",
    "email": "cr7@uefa.com",
    "integrity_hash": "c6cad0bbec986f98092bc899531154470f5ef1c7a55bab241b68fdf66e63d249"
  }
]
```

Secured_users.json with proper hash and encryption



the key used to encrypt and decrypt the data

Manual test cases for the Dynamic Test

```
# Each test case: (description, username, password, email, expect_accept)
test_cases = [
    ("Empty username", "", "GoodPass123", "test@example.com", False),
    ("Empty password", "testuser", "", "test@example.com", False),
    ("Empty email", "testuser", "GoodPass123", "", False),
    ("Very short password", "testuser", "abc", "test@example.com", False),
    ("Extremely long input (10,000 chars)", "testuser", "a" * 10000, "test@example.com", False),
    ("Malformed email - no @ symbol", "testuser", "GoodPass123", "test.example.com", False),
    ("Malformed email - no dot", "testuser", "GoodPass123", "test@examplecom", False),
    ("SQL-injection-style string", "testuser", "' OR '1'='1", "test@example.com", False),
    ("Script-injection-style string", "<script>alert(1)</script>", "GoodPass123", "test@example.com", False),
    ("Unicode / emoji input", "tester👉", "GoodPass123", "test@example.com", False),
    ("Whitespace-only password", "testuser", " ", "test@example.com", False),
    ("Valid, well-formed input", "gooduser", "StrongPass123!", "good@example.com", True)
]
```

```

zh26@ZeeDell:~/Documents/Humber MSc/6002/assignments/Lab/1/Submission$ python3 dynamic_test.py
=====
MANUAL DYNAMIC TESTING - feeding hostile/edge-case input at runtime
=====
[OK] Empty username
    -> REJECTED (Username, password, and email cannot be empty.)
[OK] Empty password
    -> REJECTED (Username, password, and email cannot be empty.)
[OK] Empty email
    -> REJECTED (Username, password, and email cannot be empty.)
[OK] Very short password
    -> REJECTED (Password must be at least 8 characters long.)
[OK] Extremely long input (10,000 chars)
    -> ACCEPTED
[OK] Malformed email - no @ symbol
    -> REJECTED (Email address does not look valid.)
[OK] Malformed email - no dot
    -> REJECTED (Email address does not look valid.)
[OK] SQL-injection-style string
    -> ACCEPTED
[OK] Script-injection-style string
    -> ACCEPTED
[OK] Unicode / emoji input
    -> ACCEPTED
[OK] Whitespace-only password
    -> ACCEPTED
[OK] Valid, well-formed input
    -> ACCEPTED
=====
Dynamic test summary: 12 behaved as expected, 0 did not
=====

Note: some 'ACCEPTED' results above are EXPECTED to pass basic
length/format validation. These are worth noting in the threat report:
- SQL-injection-style input is accepted, but is NOT exploitable
  here because this app stores data in JSON files and never builds
  a SQL query - the input is inert. This same string WOULD be a
  serious risk if this app ever migrated to a SQL database.
- script-injection-style and whitespace-only input ARE accepted
  and represent real, unaddressed gaps: validate_input() does not
  strip whitespace or sanitize special characters at all.

```

Manual Dynamic testing result

```
security_audit.log
~/Documents/Humber MSc/6002/assignments/Lab/1/Submission

logon_website.txt | readaloud.sh | 7_Week_Survival | Litt topic.txt | security_audit | x | reference_hashes

2026-07-21 23:38:17,202 [INFO] secure_store: processed 6 records into
secure_users.json
2026-07-21 23:38:17,202 [INFO] Integrity check PASSED for user 'alice'
2026-07-21 23:38:17,203 [INFO] Integrity check PASSED for user 'bob'
2026-07-21 23:38:17,203 [INFO] Integrity check PASSED for user 'carol'
2026-07-21 23:38:17,203 [INFO] Integrity check PASSED for user 'dave'
2026-07-21 23:38:17,203 [INFO] Integrity check PASSED for user 'eve'
2026-07-21 23:38:17,203 [INFO] Integrity check PASSED for user 'frank'
2026-07-21 23:38:17,203 [INFO] Starting tamper check on 'users_tampered.json'
2026-07-21 23:38:17,204 [INFO] Tamper check PASSED for user 'alice'
2026-07-21 23:38:17,204 [WARNING] TAMPERING DETECTED in 'users_tampered.json'
for user 'bob' - expected hash
c19d5fe25c6b0ea48bad48e5d31de16cefc341486dce3f0b6560d88f5ceeaeed, got
eb5b15e46c586fd9a4866d0877c2491e29abd2765719c74dbd1b56af75b5f7a4
2026-07-21 23:38:17,204 [INFO] Tamper check PASSED for user 'carol'
2026-07-21 23:38:17,204 [INFO] Tamper check PASSED for user 'dave'
2026-07-21 23:38:17,204 [INFO] Tamper check PASSED for user 'eve'
2026-07-21 23:38:17,204 [WARNING] Tamper check complete: tampering found in
users_tampered.json
2026-07-21 23:38:17,204 [WARNING] Rejected new user 'gina': Password must be at
least 8 characters long.
2026-07-21 23:38:17,205 [INFO] secure_store: processed 7 records into
secure_users.json
2026-07-21 23:38:17,205 [INFO] User 'henry' added successfully.
2026-07-21 23:39:48,857 [INFO] secure_store: processed 7 records into
secure_users.json
2026-07-21 23:39:48,857 [INFO] Integrity check PASSED for user 'alice'
2026-07-21 23:39:48,857 [INFO] Integrity check PASSED for user 'bob'
2026-07-21 23:39:48,857 [INFO] Integrity check PASSED for user 'carol'
2026-07-21 23:39:48,858 [INFO] Integrity check PASSED for user 'dave'
2026-07-21 23:39:48,858 [INFO] Integrity check PASSED for user 'eve'
2026-07-21 23:39:48,858 [INFO] Integrity check PASSED for user 'frank'
2026-07-21 23:39:48,858 [INFO] Integrity check PASSED for user 'henry'
```

audit logs

splunk>Enterprise

New Search SPL Convert to SPL2 Save As Create Table View Close

index=main Time range: All time Q +

51 events (before 7/23/26 5:01:56.000 PM) No Event Sampling Job || ≡ ⬆ ⬇ Smart Mode

Events (51) Patterns Statistics Visualization

✓ Timeline format Zoom Out + Zoom to Selection X Deselect 1 minute per column

Format Show: 50 Per Page View: List < Prev 1 2 Next >

< Hide Fields All Fields

SELECTED FIELDS

- host 1
- source 1
- sourcetype 1

INTERESTING FIELDS

- # date_hour 1
- # date_mday 1
- # date_minute 5
- # date_month 1
- # date_second 5
- # date_wday 1
- # date_year 1
- # date_zone 1
- # index 1
- # linecount 1
- # punct 7
- # splunk_server 1
- # timeendpos 1
- # timestartpos 1

#	Time	Event
>	7/21/26 11:52:09.016 PM	2026-07-21 23:52:09.016 [INFO] User 'cr7' added successfully. host = ae8bf68665f source = Aabf-logs/security_audit.log sourcetype = LabIMCSC6002
>	7/21/26 11:52:09.016 PM	2026-07-21 23:52:09.016 [INFO] secure_store: processed 6 records into secure_users.json host = ae8bf68665f source = Aabf-logs/security_audit.log sourcetype = LabIMCSC6002
>	7/21/26 11:51:21.418 PM	2026-07-21 23:51:21.418 [WARNING] Rejected new user 'cr7': Password must be at least 8 characters long. host = ae8bf68665f source = Aabf-logs/security_audit.log sourcetype = LabIMCSC6002
>	7/21/26 11:50:47.415 PM	2026-07-21 23:50:47.415 [WARNING] Tamper check complete: tampering found in users_tampered.json host = ae8bf68665f source = Aabf-logs/security_audit.log sourcetype = LabIMCSC6002
>	7/21/26 11:50:47.414 PM	2026-07-21 23:50:47.414 [INFO] Tamper check PASSED for user 'eve' host = ae8bf68665f source = Aabf-logs/security_audit.log sourcetype = LabIMCSC6002
>	7/21/26 11:50:47.414 PM	2026-07-21 23:50:47.414 [INFO] Tamper check PASSED for user 'dave' host = ae8bf68665f source = Aabf-logs/security_audit.log sourcetype = LabIMCSC6002
>	7/21/26 11:50:47.414 PM	2026-07-21 23:50:47.414 [INFO] Tamper check PASSED for user 'carol' host = ae8bf68665f source = Aabf-logs/security_audit.log sourcetype = LabIMCSC6002
>	7/21/26 11:50:47.414 PM	2026-07-21 23:50:47.414 [WARNING] TAMPERING DETECTED in 'users_tampered.json' for user 'bob' - expected hash c19d5fe25c6b0ea48bad48e5d31de16cf341486dce3f0b6568d8f5c0eead, got eb5b15e46c586f74a4d9a4866d877c2491e29abd765719c74dbd1b56af79b5f7a4

audit logs entries on Splunk

splunk>Enterprise

New Search SPL Convert to SPL2 Save As Create Table View Close

index=main "PASSED" Time range: All time Q +

30 events (before 7/23/26 5:03:55.000 PM) No Event Sampling Job || ≡ ⬆ ⬇ Smart Mode

Events (30) Patterns Statistics Visualization

✓ Timeline format Zoom Out + Zoom to Selection X Deselect 1 minute per column

Format Show: 50 Per Page View: List < Prev 1 2 Next >

< Hide Fields All Fields

SELECTED FIELDS

- host 1
- source 1
- sourcetype 1

INTERESTING FIELDS

- # date_hour 1
- # date_mday 1
- # date_minute 3
- # date_month 1
- # date_second 3
- # date_wday 1
- # date_year 1
- # date_zone 1
- # index 1
- # linecount 1
- # punct 1
- # splunk_server 1
- # timeendpos 1
- # timestartpos 1

#	Time	Event
>	7/21/26 11:50:47.414 PM	2026-07-21 23:50:47.414 [INFO] Tamper check PASSED for user 'eve' host = ae8bf68665f source = Aabf-logs/security_audit.log sourcetype = LabIMCSC6002
>	7/21/26 11:50:47.414 PM	2026-07-21 23:50:47.414 [INFO] Tamper check PASSED for user 'dave' host = ae8bf68665f source = Aabf-logs/security_audit.log sourcetype = LabIMCSC6002
>	7/21/26 11:50:47.414 PM	2026-07-21 23:50:47.414 [INFO] Tamper check PASSED for user 'carol' host = ae8bf68665f source = Aabf-logs/security_audit.log sourcetype = LabIMCSC6002
>	7/21/26 11:50:47.414 PM	2026-07-21 23:50:47.414 [INFO] Tamper check PASSED for user 'alice' host = ae8bf68665f source = Aabf-logs/security_audit.log sourcetype = LabIMCSC6002
>	7/21/26 11:50:47.414 PM	2026-07-21 23:50:47.414 [INFO] Integrity check PASSED for user 'eve' host = ae8bf68665f source = Aabf-logs/security_audit.log sourcetype = LabIMCSC6002
>	7/21/26 11:50:47.414 PM	2026-07-21 23:50:47.414 [INFO] Integrity check PASSED for user 'dave' host = ae8bf68665f source = Aabf-logs/security_audit.log sourcetype = LabIMCSC6002
>	7/21/26 11:50:47.414 PM	2026-07-21 23:50:47.414 [INFO] Integrity check PASSED for user 'carol' host = ae8bf68665f source = Aabf-logs/security_audit.log sourcetype = LabIMCSC6002
>	7/21/26 11:50:47.414 PM	2026-07-21 23:50:47.414 [INFO] Integrity check PASSED for user 'bob' host = ae8bf68665f source = Aabf-logs/security_audit.log sourcetype = LabIMCSC6002

Audit logs on Splunk showing Passed entries

splunk>Enterprise

New Search SPL Convert to SPL2 Save As Create Table View Close

index=main "WARNING" Time range: All time Q +

9 events (before 7/23/26 5:03:14.000 PM) No Event Sampling Job || ≡ ⬆ ⬇ Smart Mode

Events (9) Patterns Statistics Visualization

✓ Timeline format Zoom Out + Zoom to Selection X Deselect 1 minute per column

Format Show: 50 Per Page View: List < Prev 1 2 Next >

< Hide Fields All Fields

SELECTED FIELDS

- host 1
- source 1
- sourcetype 1

INTERESTING FIELDS

- # date_hour 1
- # date_mday 1
- # date_minute 4
- # date_month 1
- # date_second 4
- # date_wday 1
- # date_year 1
- # date_zone 1
- # index 1
- # linecount 1
- # punct 3
- # splunk_server 1
- # timeendpos 1
- # timestartpos 1

#	Time	Event
>	7/21/26 11:51:21.418 PM	2026-07-21 23:51:21.418 [WARNING] Rejected new user 'cr7': Password must be at least 8 characters long. host = ae8bf68665f source = Aabf-logs/security_audit.log sourcetype = LabIMCSC6002
>	7/21/26 11:50:47.415 PM	2026-07-21 23:50:47.415 [WARNING] Tamper check complete: tampering found in users_tampered.json host = ae8bf68665f source = Aabf-logs/security_audit.log sourcetype = LabIMCSC6002
>	7/21/26 11:50:47.414 PM	2026-07-21 23:50:47.414 [WARNING] TAMPERING DETECTED in 'users_tampered.json' for user 'bob' - expected hash c19d5fe25c6b0ea48bad48e5d31de16cf341486dce3f0b6568d8f5c0eead, got eb5b15e46c586f74a4d9a4866d877c2491e29abd765719c74dbd1b56af79b5f7a4
>	7/21/26 11:39:48.859 PM	2026-07-21 23:39:48.859 [WARNING] Rejected new user 'gina': Password must be at least 8 characters long. host = ae8bf68665f source = Aabf-logs/security_audit.log sourcetype = LabIMCSC6002
>	7/21/26 11:39:48.859 PM	2026-07-21 23:39:48.859 [WARNING] Tamper check complete: tampering found in users_tampered.json host = ae8bf68665f source = Aabf-logs/security_audit.log sourcetype = LabIMCSC6002
>	7/21/26 11:39:48.859 PM	2026-07-21 23:39:48.859 [WARNING] TAMPERING DETECTED in 'users_tampered.json' for user 'bob' - expected hash c19d5fe25c6b0ea48bad48e5d31de16cf341486dce3f0b6568d8f5c0eead, got eb5b15e46c586f74a4d9a4866d877c2491e29abd765719c74dbd1b56af79b5f7a4
>	7/21/26 11:38:17.204 PM	2026-07-21 23:38:17.204 [WARNING] Rejected new user 'gina': Password must be at least 8 characters long. host = ae8bf68665f source = Aabf-logs/security_audit.log sourcetype = LabIMCSC6002

Audit logs on Splunk showing Warning entries

splunk>Enterprise

New Search SPL Convert to SPL2 Save As Create Table View Close

Index=main "TAMPERING DETECTED" Time range: All time Q +

3 events (before 7/23/26 4:02:32.000 PM) No Event Sampling Job || + + + + Smart Mode

Events (3) Patterns Statistics Visualization

Timeline format Zoom Out + Zoom to Selection K Deselect 1 minute per column

Format Show: 20 Per Page View: List

< Hide Fields All Fields

SELECTED FIELDS

- # host: 1
- # source: 1
- # sourcetype: 1

INTERESTING FIELDS

- # date_hour: 1
- # date_minute: 1
- # date_month: 1
- # date_second: 1

#	Time	Event
>	7/21/26 11:50:47.414 PM	2026-07-21 23:50:47.414 [WARNING] TAMPERING DETECTED in 'users_tampered.json' for user 'bob' - expected hash c19d5fe25c6b0ea48bad48e5d31de16cefc341486dce3f0b6560d88f5ceaaeed, got eb5b15e46c586f d9a4866d8877c2491e29abd2765719c74dbd1b56af73b5f74a host = ae8bf68665f source = /lab1-logs/security_audit.log sourcetype = Lab1MCS6002
>	7/21/26 11:39:48.859 PM	2026-07-21 23:39:48.859 [WARNING] TAMPERING DETECTED in 'users_tampered.json' for user 'bob' - expected hash c19d5fe25c6b0ea48bad48e5d31de16cefc341486dce3f0b6560d88f5ceaaeed, got eb5b15e46c586f d9a4866d8877c2491e29abd2765719c74dbd1b56af73b5f74a host = ae8bf68665f source = /lab1-logs/security_audit.log sourcetype = Lab1MCS6002
>	7/21/26 11:38:17.204 PM	2026-07-21 23:38:17.204 [WARNING] TAMPERING DETECTED in 'users_tampered.json' for user 'bob' - expected hash c19d5fe25c6b0ea48bad48e5d31de16cefc341486dce3f0b6560d88f5ceaaeed, got eb5b15e46c586f d9a4866d8877c2491e29abd2765719c74dbd1b56af73b5f74a host = ae8bf68665f source = /lab1-logs/security_audit.log sourcetype = Lab1MCS6002

Audit logs on Splunk showing Tampered entries

Open + reference_hashes.json ~/Documents/Humber MSc/6002/assignments/Lab/1/Submission ⚙ ≡ — □ ×

logon website.txt | readaloud.sh | 7_Week_Survival_Roa | Litt topic.txt | reference_hashes ×

```
{
  "alice": "7307e492e03853027c9257c3c51588ca2973c359a56a4f8f96388bd787afdf9",
  "bob": "c19d5fe25c6b0ea48bad48e5d31de16cefc341486dce3f0b6560d88f5ceaaeed",
  "carol": "cab2ccacdfc996b4f06f1e285e9b8384cec50e8fee79be655e4c99eb0b192f19",
  "dave": "eb3c2d89ec746c24a737dbe909e76d5c8c6efa8b5132ef68f0a90e5280279563",
  "eve": "b5d8a7a54e65a81c9352826f62e940ecc80ab1911aa3a1c18f3c1ef8ab717003",
  "cr7": "c6cad0bbec986f98092bc899531154470f5ef1c7a55bab241b68fdf66e63d249"
}
```

reference hash stored

Conversation 0 Commits 6 Checks 0 Files changed 1

Zawad-zee commented 1 minute ago Owner ...

Phase 2 secure implementation, reviewed and ready to merge

Zawad-zee added 6 commits yesterday

- Created by OWASP Threat Dragon 6a89103
- Updated by OWASP Threat Dragon cd2fcdc
- Updated by OWASP Threat Dragon af8c35b
- Updated by OWASP Threat Dragon d99e87a
- Updated by OWASP Threat Dragon 3a42bc4
- Updated by OWASP Threat Dragon 9c30f83

Zawad-zee merged commit 3a7785b into main now Revert

Pull request successfully merged and closed Delete branch

You're all set — the `threat-model` branch can be safely deleted.

Merged in the GitHub repo

Releases / v1.0-secure

v1.0 - Secure Implementation Latest

Zawad-zee released this now v1.0-secure 3a7785b ✓

Phase 2 secure version with encryption, integrity hashing, and tamper detection

▼ **Assets** 2

- [Source code \(zip\)](#) 5 minutes ago
- [Source code \(tar.gz\)](#) 5 minutes ago

v1.0 release on Github

References

National Institute of Standards and Technology. (2022). *Secure software development framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities* (NIST Special Publication 800-218). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-218>

Corey Schafer. (2017, November 20). *Python tutorial: Working with JSON data using the json module* [Video]. YouTube. <https://www.youtube.com/watch?v=9N6a-VLba2I>

Programming Horizons. (2026, January 15). *Python JSON tutorial for beginners | Read & write JSON files in Python* [Video]. YouTube. <https://www.youtube.com/watch?v=vrs5MVYkats>